المملكة العربية السعودية
وزارة التعليم العالي
جامعة الإمام محمد بن سعود الإسلامية
كلية علوم الحاسب والمعلومات
قسم علوم الحاسب
منتظم



المقرر: عال ٥١٢
العام الجامعي: ١٤٣٥-١٤٣٤
المدة: ١٢٠ دقيقة

Final Exam
Course: CS215 Design and Analysis of Algorithms
Dec 29th, 2013 10:00 to 12:00

Instructor:	
Student Name:	Level:
Student Number:	Section:

Question	Points	Score
Question 1	5	
Question 2	10	
Question 3	8	
Question 4	8	
Question 5	9	
Bonus Question	0	
Total:	40	

1. This is a closed book exam. Do not use any notes or books!
2. Show your work. Partial credit will be given. Grading will be based on correctness, clarity and neatness.
3. We suggest you to read the whole exam before beginning to work any problem.

Question 1: ..... **5 points**

(a) (**2 points**) Complete the following table. MST = Minimum Spanning Tree.

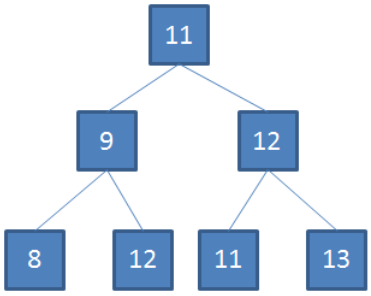
Algorithm	Worst Case Complexity
0-1 Knapsack Problem using Dynamic Programming	
0-1 Knapsack Problem using Brute Force	
Breadth First Search	
Depth First Search	
Kruskal's algorithm for finding MST	
Prim's algorithm for finding MST	
Shortest path by Dijkstra	
Shortest path by Bellman Ford	

Solution:

0.25 points each

Algorithm	Worst Case
0-1 Knapsack Problem using Dynamic Programming	$O(nW) or O(n * Complexity)$
0-1 Knapsack Problem using Brute Force	$O(2^n)$
Breadth First Search	$O(V+E)$
Depth First Search	$O(V+E)$
Kruskal's algorithm for finding MST	$O(ElogV)$
Prim's algorithm for finding MST	$O(ElogV)$
Shortest path by Dijkstra	$O((V+E)logV)$
Shortest path by Bellman Ford	$O(V+E)$

(b) (**3 points**) Use Pre-order tree walk, In-order tree walk and Post-order tree walk to print all keys in sorted order for the Binary Tree below



Solution:

pre-order tree walk: 11,9,8,12,12,11,13 ..... 1 Point
in-order tree walk: 8,9,12,11,11,12,13 ..... 1 Point
post-order tree walk: 8,12,9,11,13,12,11 ..... 1 Point

Question 2: ..... **10 points**

Consider the following instance of the 0,1 knapsack problem

Item	1	2	3	4	5
Benefit	6	12	4	9	14
Weight	1	5	3	4	6

The maximum allowable total weight in the knapsack is $W = 10$.

Find an optimal solution for the above problem with the weights and benefits above using Dynamic Programming. Be sure to state both the value of the maximum benefit that you obtain as well as the item(s) that you need to obtain this benefit. Show all steps.

Solution:

Create Value table

Value	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	6	6	6	6	6	6	6	6	6	6
2	0	6	6	6	6	12	18	18	18	18	18
3	0	6	6	6	10	12	18	18	18	22	22
4	0	6	6	6	10	15	18	18	19	22	27
5	0	6	6	6	10	15	18	20	20	22	27

Maximum Benefit = Value[5][10] = 27

Create keep table for finding items

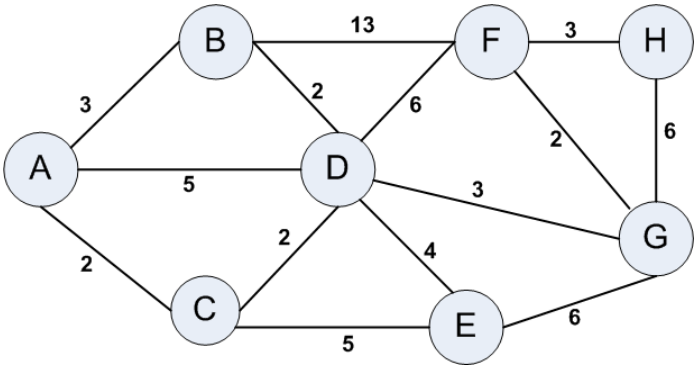
Keep	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1
2	0	0	0	0	0	1	1	1	1	1	1
3	0	0	0	0	1	0	0	0	0	1	1
4	0	0	0	0	0	1	0	0	1	0	1
5	0	0	0	0	0	0	0	1	1	0	0

- Since keep[5,10] == 0, we do not keep this item
- Now, we look for keep[4,10] = 1. so we keep it and reduce the weight as follows W = 10 - 4 = 6
- keep[3,6] = 0, we do not keep it
- keep[2,6] = 1, we weep it and W = 6 - 5 = 1
- keep[1,1] = 1, we keep it and this will end the back tracing

Item selected 1, 2, 4

Question 3: ..... **8 points**

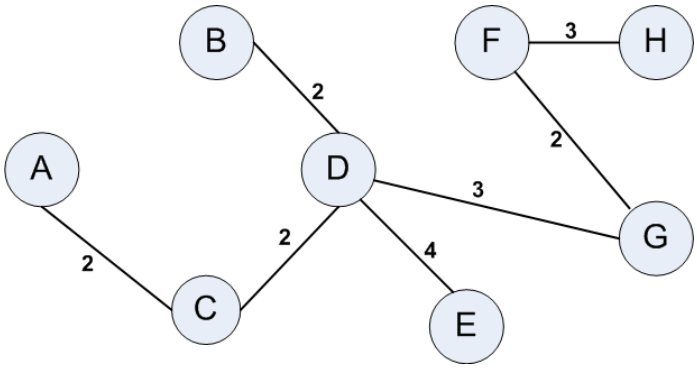
Find The Minimum Spanning Tree of the following graph using Kruskal's algorithm. Show the steps and the MST along with resulting length.



Solution:

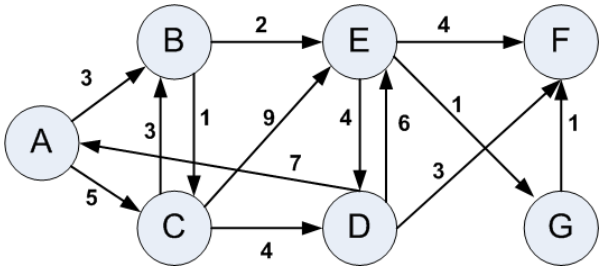
Edge	Included	Weight
AC	Yes	2
BD	Yes	2
CD	Yes	2
FG	Yes	2
AB	No (Loop)	3
DG	Yes	3
FH	Yes	3
DE	Yes	4
AD	No	5
CE	No	5
EG	No	6
DF	No	6
GH	No	6
BF	No	13

The length of the MST is 18.



Question 4: ..... **8 points**

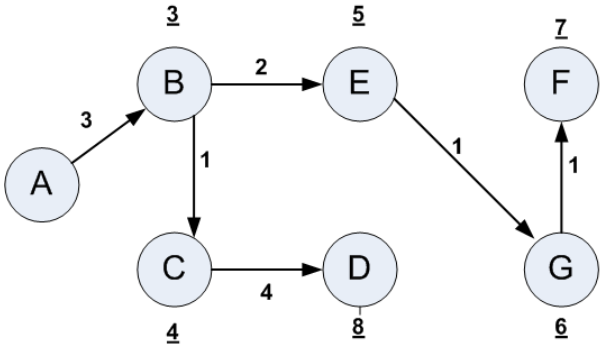
Consider the following Graph



Find the shortest path from source A to all destinations (nodes) using Dijkstra's algorithm. Show all steps, tables and the resulting path.

Solution:

Q	A	B	C	D	E	F	G	S
	0	∞	∞	∞	∞	∞	∞	{}
		3	5	∞	∞	∞	∞	{A}
			4	∞	5	∞	∞	{A,B}
				8	**5**	∞	∞	{A,B,C}
				8		9	**6**	{A,B,C,E}
				8		**7**		{A,B,C,E,G}
				8				{A,B,C,E,F,G}
								{A,B,C,D,E,F,G}



Question 5: ..... **9 points**

(a) (**5 points**) Explain what are P, NP and NP-Complete problems. What is meant by "is P = NP" ?

Solution:
"P" – Problems that can be solved in polynomial time on a deterministic machine.
"NP" – Problems that cannot be solved in polynomial time on a deterministic machine but their solution can be verified in polynomial time on a deterministic machine. Alternatively, problems that can be solved in polynomial time on a non-deterministic machine.
"NP-Complete" – A problem is NP-Complete if it belongs to class NP and all NP problems can be reduced to it in polynomial time.
"is P = NP" – it means whether an NP problem can belong to class P problem. In other words whether every problem whose solution can be verified by a computer in polynomial time can also be solved by a computer in polynomial time.

(b) Suppose that A and B, and that A $\leq p$B i.e. A can be reduced to B in polynomial time. Circle one answer for each statement below.

i. (**1 point**) If A $\in$ NP-Complete and B is in NP then B $\in$ NP-Complete
   A. True
   B. False

ii. (**1 point**) If B $\in$ P, then A $\in$ P
   A. True
   B. False

iii. (**1 point**) If A is NP-hard then B is NP-hard
   A. True
   B. False

iv. (**1 point**) If B is NP-hard then A is NP-hard
   A. True
   B. False

6. (a) (2 points (bonus)) Let G = (V, E) be a weighted, directed graph with exactly one negative weight edge and no negative-weight cycles. Give an algorithm to find the shortest distance from s to all other vertices in V that has the same running time as Dijkstra.

Solution:

Let's say the negative-weight edge is (u; v). First, remove the edge and run Dijkstra from s. Then, check if ds[u] + w(u,v) ¡ ds[v]. If not, then we're done. If yes, then run Dijkstra from v, with the negative-weight edge still removed. Then, for any node t, its shortest distance from s will be min(ds[t], ds[u] + w(u, v) + dv[t]).

(b) (4 points (bonus)) An independent set in a Graph is defined as an independent set of vertices in a graph, no two of which are adjacent. That is, it is a set S of vertices such that for every two vertices in S, there is no edge connecting the two. The size of an independent set is the number of vertices it contains. Assume that independent set problem is in NP.

Assume the following 3-CNF [1] which is NP-Complete.

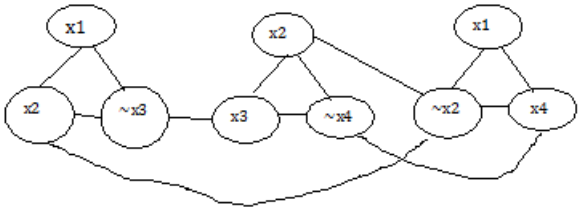
(x1 $\vee$ x2 $\vee$ $\sim$x3) $\wedge$ (x2 $\vee$ x3 $\vee$ $\sim$x4) $\wedge$ (x1 $\vee$ $\sim$x2 $\vee$ x4) where $\sim$ means NOT function, $\vee$ means OR function and $\wedge$ means AND function.

Show that Independent set in a graph corresponding to the 3-CNF is NP-Complete. What is the size of the Independent set S (i.e. maximum number of nodes in S that are not adjacent)? Explain your answer.

[Hint: Reduce 3-CNF to a graph. You can consider a node (e.g. x2) is connected to ($\sim$x2) etc.]

Solution:

Since 3-CNF is an NP-Complete problem and Independent Set problem is in NP, we just need to reduce the 3-CNF to Independent Set problem. We also need to show that the maximum size of the corresponding Independent Set is 3.



Since 3-CNF is satisfiable, there is at least one literal (variable) that is true in each clause. Let S be a set of one such True literal from each clause. Thus $|S|$ = k = 3. And no two nodes in S are connected by an edge. Note that this is the maximum value for k.

If the graph has an Independent set S of size k, we know that it has one node from each "clause triangle". Set those terms to True. This is possible because no two nodes are negations of each other.

Thus, we have reduced the 3-CNF to a graph which has maximum Independent set of size k=3 (example: for x1 = x2 = x4 = 1, we have these nodes in the Independent set. Note that in this case, x2 causes the middle "clause triangle" to become true etc).

[1] See Appendix for its definition

A Appendix

Literal

A literal is either a (boolean) variable or its negation. Example, x or $\sim$x.

Clause

A clause is the logical OR of one or more literals.

CST Problem

The circuit-satisfiability problem (CST) is, "Given a boolean combinational circuit composed of AND, OR, and NOT gates, is it satisfiable?". Remember, A truth assignment for a boolean combinational circuit is a set of boolean input values. We say that a one-output boolean combinational circuit is **satisfiable** if it has a satisfying assignment: a truth assignment that causes the output of the circuit to be 1. The circuit-satisfiability problem arises in the area of computer-aided hardware optimization. The circuit-satisfiability problem is NP-complete.

3-CST Problem

A 3-CNF expression is a boolean expression in CNF such that each clause is a disjunction of exactly **3** literals.